Product Requirements Document (PRD)

Project: Legal Document RAG Assistant (TCS AI Internship Assignment)

1. Executive Summary

Objective: To design and build a Retrieval-Augmented Generation (RAG) application that accurately answers user queries based strictly on the provided sections of Public Law 111-5 (HITECH Act, Sections 13407 and 13424). **Key Constraint:** The application must operate entirely on free/open-source models, utilize a Chroma vector data store, and rigidly refuse to answer out-of-context questions by returning an exact fallback phrase.

2. Scope & Target Audience

- **In Scope:** * Ingestion and parsing of the specific 6-page PDF document.
  - Vectorization of document chunks using a free, locally hosted embedding model.
  - Storing embeddings in a local Chroma vector database.
  - Querying a free-tier or locally hosted Large Language Model (LLM) to generate answers.
  - Strict implementation of context guardrails.
- **Out of Scope:** General knowledge answering, internet search capabilities, or paid API integrations (e.g., OpenAI GPT-4, Pinecone).

3. System Architecture & Tech Stack

To satisfy the requirement of zero cost, the following stack is mandated:

- **Programming Language:** Python 3.10+
- **Framework:** LangChain or LlamaIndex (for orchestrating the ingestion, retrieval, and LLM chains).
- **Vector Database: Chroma (ChromaDB)** running locally.
- **Embedding Model (Encoder):** `BAAI/bge-small-en-v1.5` or `sentence-transformers/all-MiniLM-L6-v2` . Both are completely free, run locally via Hugging Face, and are highly optimized for RAG semantic search.
- **Generator Model (LLM):** * *Option A (Cloud API - Recommended for low hardware specs):* **Llama-3-8B-Instruct** or **Mistral-7B** via a generous free-tier API provider (e.g., **Groq**, **GitHub Models Free Tier**, or **Hugging Face Serverless Inference**).

    - *Option B (Local execution):* **Ollama** running `llama3` or `mistral` (Requires an environment with at least 8GB RAM/VRAM).
- **Frontend UI (Optional but recommended):** Streamlit or Gradio.

4. Functional Requirements

4.1 Document Ingestion & Chunking

- **Parser:** Use `PyPDFLoader` to extract raw text from the provided PDF.
- **Chunking Strategy:** Implement a `RecursiveCharacterTextSplitter`.
    - *Chunk Size:* ~500 tokens.
    - *Chunk Overlap:* ~100 tokens.
    - *Rationale:* Legal texts contain highly connected clauses. An overlap ensures that definitions (e.g., "unsecured PHR identifiable health information") are not separated from their corresponding context.

4.2 Retrieval & Storage

- **Vectorization:** Convert all chunks into dense vectors using the selected Hugging Face encoder model.
- **Storage:** Persist the vectors and their text metadata into a local ChromaDB collection.
- **Retrieval Mechanism:** Upon receiving a user query, convert it using the same encoder and perform a semantic similarity search (Cosine Similarity) in Chroma to retrieve the Top-K (e.g., K=3) most relevant chunks.

4.3 Generation & Guardrails (Strict Fallback)

- **Contextual Grounding:** The retrieved chunks must be passed into the LLM context window.
- **Fallback Rule:** If the question is outside the scope of the retrieved context, the app must output exactly: **"Sorry I can't find anything relevant to this question"**.

5. Error Minimization & Accuracy Optimization (Inference)

To prevent hallucinations and minimize inference errors, the following techniques must be implemented:

1. **Pre-LLM Similarity Thresholding (Crucial):**
    - *Technique:* When querying Chroma, evaluate the similarity score/distance of the top retrieved chunks.

- *Action:* If the best matching chunk has a distance score indicating low relevance (e.g., Cosine Distance > 0.4, depending on the embedding model), **short-circuit the pipeline**. Do not send the prompt to the LLM. Instead, immediately return the fallback string. This guarantees zero hallucinations for wildly out-of-domain questions.

2. **Strict System Prompt Engineering:**
   - *Technique:* Confine the LLM's behavior heavily in the system prompt.
   - *Template Example:*

```
You are a strict legal assistant. You must answer the user's question usi
If the answer is not explicitly contained in the Context, do not attempt
Instead, you must reply with exactly this phrase and nothing else: "Sorry

Context: {context}
Question: {question}
```

3. **Maximum Marginal Relevance (MMR) Search:**
   - *Technique:* Instead of standard similarity search, configure Chroma to use MMR.
   - *Rationale:* MMR optimizes for both relevance to the query *and* diversity among the retrieved chunks. If a user asks a broad question (e.g., "What are all the reports the Secretary must submit?"), MMR ensures the retriever fetches diverse parts of the document rather than duplicate clauses, minimizing incomplete answers.
4. **Temperature Control:**
   - *Technique:* Set the LLM's `temperature` parameter to `0.0` or `0.1`.
   - *Rationale:* A low temperature removes creative randomness, forcing the model to be deterministic, factual, and strictly reliant on the provided text.

6. Acceptance Criteria

- [ ] Application successfully parses and stores the provided HITECH Act PDF in Chroma.
- [ ] Application accurately answers complex contextual questions (e.g., "What is the definition of Breach of Security?").
- [ ] Application outputs the exact string `"Sorry I can't find anything relevant to this question"` for out-of-domain queries (e.g., "What is the capital of France?" or "Explain general HIPAA privacy rules.").